

UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA
FACULTAD DE SISTEMAS Y TELECOMUNICACIONES
CARRERA DE INGENIERÍA EN SOFTWARE

ASIGNATURA:
SISTEMAS DISTRIBUIDOS Y CLOUD COMPUTING

TEMA:

AUDITORÍA MANUAL — RED SOCIAL DISTRIBUIDA (2026)

Arena de Auditoría Técnica · Fase 1 (Red Team)

ELABORADO POR (RED TEAM):

Andy Bryan Alejandro Vera
Alisson Yamel Reyes Ricardo
Gino Maximiliano Bermúdez Santos
Yandris Miguel Rivera Torres

CURSO Y PARALELO: SOFTWARE 7/1

DOCENTE: ING. ANTHONY ABRAHAN PACHAY ESPINOZA

LA LIBERTAD – ECUADOR
Noviembre 2026

Índice

Resumen Ejecutivo **2**

1. Tabla Consolidada de Hallazgos **3**

2. Misión 1 — FROM ZERO: Reproducibilidad **5**
2.1. 1.1 Resumen ejecutivo . . . 5
2.2. 1.2 Pasos realizados . . . 5
2.3. 1.3 Hallazgos . . . 5
2.4. 1.4 Conclusión de la misión . . . 6

3. Misión 2 — REQUIREMENTS HUNT **6**
3.1. 2.1 Resumen ejecutivo . . . 6
3.2. 2.2 Tabla de cumplimiento . . . 6
3.3. 2.3 Verificación automática del seed . . . 6
3.4. 2.4 Hallazgos . . . 8
3.5. 2.5 Conclusión de la misión . . . 8

4. Misión 3 — CODE AUTOPSY **8**
4.1. 3.1 Resumen ejecutivo . . . 8
4.2. 3.2 Acoplamiento sano (lo que está bien) . . . 8
4.3. 3.3 Acoplamiento que va a doler . . . 9
4.4. 3.4 Conclusión de la misión . . . 10

5. Misión 4 — BREAK IT: Seguridad y pentesting controlado **11**
5.1. 4.1 Resumen ejecutivo . . . 11
5.2. 4.2 Hipótesis → Prueba → Evidencia → Impacto . . . 11
5.3. 4.3 Lo que SÍ está bien . . . 14
5.4. 4.4 Conclusión de la misión . . . 14

6. Misión 5 — UX UNDER FIRE **14**
6.1. 5.1 Resumen ejecutivo . . . 14
6.2. 5.2 Flujos evaluados . . . 15
6.2.1. 5.2.1 Registro . . . 15
6.2.2. 5.2.2 Login . . . 15
6.2.3. 5.2.3 Crear publicación . . . 15
6.2.4. 5.2.4 Seguir a un usuario . . . 15
6.2.5. 5.2.5 Comentar . . . 15
6.2.6. 5.2.6 Chat . . . 16
6.2.7. 5.2.7 Activar notificaciones . . . 16
6.3. 5.3 Hallazgos específicos . . . 16
6.4. 5.4 Lo que SÍ está bien . . . 17
6.5. 5.5 Conclusión de la misión . . . 17

7. Misión 6 — CHAOS MODE **17**
7.1. 6.1 Resumen ejecutivo . . . 17
7.2. 6.2 Modos de fallo analizados . . . 17
7.2.1. 6.2.1 Neo4j se detiene temporalmente . . . 17
7.2.2. 6.2.2 Redis se detiene . . . 18
7.2.3. 6.2.3 Backend-1 se reinicia . . . 18
7.2.4. 6.2.4 Neo4j se corrompe (constraint violation masiva) . . . 18
7.2.5. 6.2.5 Token JWT expira . . . 18
7.2.6. 6.2.6 Acciones concurrentes (reaccionar dos veces al mismo post rápido) . . . 18
7.2.7. 6.2.7 Frontend con un servicio no disponible (Neo4j caído) . . . 19
7.3. 6.3 Hallazgos . . . 19
7.4. 6.4 Lo que SÍ está bien . . . 19
7.5. 6.5 Conclusión de la misión . . . 19

Anexo A — Hallazgos del equipo original (mapeados a las misiones) **20**

Anexo B — Hallazgos propios que el equipo original no tenía **21**

Anexo C — Resumen de la auditoría manual del equipo **22**

Conclusiones y Siguientes Pasos **24**

Asignatura: Sistemas Distribuidos y Cloud Computing · **Curso:** Software 7/1
Docente: ING. ANTHONY ABRAHAN PACHAY ESPINOZA
Equipo Auditor (Red Team): Andy Bryan Alejandro Vera, Gino Maximiliano Bermúdez Santos, Alisson Yamel Reyes Ricardo, Yandris Miguel Rivera Torres

Modalidad: Revisión cruzada independiente (Fase 1 · Red Team vs Blue Team)
Repositorio auditado: `red-social-distribuida` (Frontend React 19 + Backend Quarkus 3.33 LTS en Java 21, Neo4j 5, Redis 7, MinIO)
Alcance y modo de ejecución: Análisis estático de código + dinámico con Docker Compose. Misión 1 (reproducibilidad) ejecutada de forma autónoma desde cero. Misiones 2 a 6 evaluadas con inspección exhaustiva de código, configuración de infraestructura y trazabilidad de fallos.

Resumen Ejecutivo

Misión	Resultado	Severidad máxima	Veredicto
[M1] M1 — Reproducibilidad (from zero)	Reproducible	Media	El docker compose up -d –build levanta todo. Tarda la primera vez por compilación de MinIO desde código.
[M2] M2 — Requirements Hunt	Cumple (con gaps UX)	Media	Las 8 consultas Cypher (C1–C8) están y se verifican al final del seed. Cubre registro, login, perfil, follow, posts con imagen, feed, reacciones, comentarios con hilos, recomendaciones, chat en tiempo real y Web Push.
[M3] M3 — Code Autopsy	Aprobado con reservas	Alta	Backend bien estructurado por capas (Resource → Service → Repository), con inyección por constructor en la mayoría de las clases. Acoplamiento de contenido (PostRepository.PUBLIC_FIELDS consume CommentRepository.COUNT_OF_POST), PostService con 5 dependencias, ChatSocket con field injection y un AuthService → UserService → UserRepository que acopla transversalmente. Spanglish estructural.
[M4] M4 — Seguridad	Aprobado con reservas críticas	Crítica	Permisos 644 sobre privateKey.pem dentro del volumen app_keys (CRÍTICO), user enumeration por timing en AuthService.login, localStorage para JWT sin httpOnly (aceptable, pero documentado), sin Content-Security-Policy en Nginx, 401 desloguea sin distinguir vencido vs inválido.
[M5] M5 — UX Under Fire	Aprobado con fricción	Alta	Flujo de registro es de dos round-trips (registro + login). Drafts de PostComposer se pierden al refrescar. Branding inconsistente (ZENIT en la UI, nodouni en el SW). Cubre accesibilidad básica (skip-link, aria-label, aria-pressed).
[M6] M6 — Chaos Mode	Buena resiliencia, con gaps	Media	RedisChatBroker reconecta con backoff, useChatConnection reintenta con backoff exponencial 1s → 30s, chat.history reintenta 3 veces ante 502/503/504/0. Faltan tests de: token JWT expirado, race condition en POST /auth/registro concurrente, recovery de reconexión WS con mensajes en vuelo.

Cuadro 1: Resumen Ejecutivo por Misión

Veredicto global: El sistema es publicable en su estado actual para una demo académica. Antes de cualquier uso más allá del aula, se debe corregir el permiso chmod 644 sobre privateKey.pem (Crítico) y agregar Content-Security-Policy en Nginx (Alta). El resto es hardening progresivo.

1 Tabla Consolidada de Hallazgos

[redacted]	Misión	Severidad	Hallazgo (resumen)	Evidencia clave
H-01	M1	Media	El primer arranque compila MinIO desde código fuente; tarda más de lo que sugiere el README.	`docker-compose.yml` → minio con build: { context: ./infra/minio, target: minio }.
H-02	M1	Baja	Si el operador borra el volumen app_keys, todos los JWT emitidos y las suscripciones push dejan de ser válidos (documentado en README, pero sin alerta de arranque).	`docker-compose.yml` línea 96 (chmod 644 /keys/*) y keys-init job.
H-03	M2	Baja	El feed muestra comentarios que pueden ser mayores que el hilo visible (200), sin advertencia al usuario.	`backend/src/main/java/com/redsocial/comentarios/CommentRepository.java:23` MAX_THREAD_COMMENTS = 200.
H-04	M2	Media	No hay bandeja de notificaciones en la UI; el push solo se entrega cuando el Service Worker la muestra, no hay historial dentro de la app.	`frontend/src/pages/SettingsPage.tsx` (no hay componente de bandeja), `sw.js` solo muestra notification.
H-05	M3	Alta	PostService recibe 5 dependencias (repository, storage, images, events, assembler) y create() orquesta validación, upload a S3, commit a Neo4j, evento y ensamblado. Buen momento para partir en PostCreator + colaboradores.	`backend/src/main/java/com/redsocial/posts/PostService.java:21-26`.
H-06	M3	Media	PostRepository.PUBLIC_FIELDS consume CommentRepository.COUNT_OF_POST (constante compartida entre paquetes). Cambias comentarios y recompilas posts.	`backend/src/main/java/com/redsocial/posts/PostRepository.java:23-29`.
H-07	M3	Media	AuthService depende de UserService (servicio) en lugar de UserRepository (repositorio). Acoplamiento transitivo: un cambio en UserService.profile() puede romper AuthService.register() sin tocar auth.	`backend/src/main/java/com/redsocial/auth/AuthService.java:23-26`, 32-49.
H-08	M3	Media	ChatSocket usa field injection (@Inject en 4 campos) en lugar de constructor injection. El resto del backend migró a constructor; este quedó atrás.	`backend/src/main/java/com/redsocial/chat/ChatSocket.java:31-38`.
H-09	M3	Baja	Spanglish estructural: paquetes en español (comentarios, notificaciones) + métodos en inglés (create, thread, save, history) + mensajes al usuario en español (mensaje no message). Búsqueda con grep se vuelve adivinar el idioma.	`backend/src/main/java/com/redsocial/notificaciones/ClavePublica.java`, CommentRequest(texto, respondeA), etc.
H-10	M4	Crítica	La clave privada del JWT queda legible por todos: `docker-compose.yml` ejecuta chmod 644 /keys/* y monta app_keys: /keys: ro en ambos backends. 644 permite que cualquier proceso del host con acceso al volumen pueda firmar tokens. Debería ser 600 o 640.	`docker-compose.yml:102` (chmod 644 /keys/*).
H-11	M4	Alta	User enumeration por timing en AuthService.login(): cuando el usuario no existe, el Optional se filtra sin correr BcryptUtil.matches. La rama "no existe" termina antes que "existe + contraseña mala", detectable por latencia.	`backend/src/main/java/com/redsocial/auth/AuthService.java:51-58` y `AuthRepository.java:51-66`.

[redacted]	Misión	Severidad	Hallazgo (resumen)	Evidencia clave (cont.)
H-12	M4	Alta	Sin Content-Security-Policy en Nginx. Solo se envían X-Content-Type-Options, X-Frame-Options, Referrer-Policy. Un XSS básico en comentarios podría cargar scripts arbitrarios desde cualquier origen.	`frontend/nginx.conf` líneas de add_header (sin CSP).
H-13	M4	Media	localStorage para JWT (sin httpOnly) implica que cualquier XSS roba el token. Aceptable con JWT de 24h y sin refresh, pero NO guardar datos sensibles.	`frontend/src/lib/session.ts:3` SESSION_KEY = "nodouni.session".
H-14	M4	Media	El frontend desloguea en cualquier 401 sin distinguir "vencido" vs "inválido". Un request viejo con token expirado que se cruza con un refresh, o skew de reloj, desloguea al usuario sin aviso.	`frontend/src/lib/api.ts:40-41` (if (response.status === 401 && token && getSession()?.token === token) setSession(null)).
H-15	M4	Media	ImageValidator solo acepta png/jpeg/gif. Celulares modernos sacan WebP/HEIC por defecto; el usuario tiene que convertir antes de subir.	`backend/src/main/java/com/redsocial/media/ImageValidator.java:22-25` (TYPES = Map.of("png", "jpeg", "gif")).
H-16	M4	Media	VAPID_SUBJECT por defecto es mailto:admin@example.com y NEO4J_PASSWORD por defecto es devpassword. No hay un check de arranque que falle si esos valores siguen en prod.	`docker-compose.yml` (VAPID_SUBJECT: ${VAPID_SUBJECT:-mailto:admin@example.com}), `application.properties`.
H-17	M4	Media	Sin observabilidad distribuida: no hay Micrometer/OpenTelemetry ni X-Request-Id propagado por Nginx. Con round robin, no se sabe qué instancia atendió un fallo reportado por el usuario.	`application.properties` (no hay quarkus-micrometer ni quarkus-opentelemetry), `frontend/nginx.conf` (no propaga X-Request-Id).
H-18	M5	Alta	El registro es de dos round-trips: POST /auth/registro devuelve Profile, no token; el frontend después llama POST /auth/login con la misma contraseña. Son 2 requests + 2 consumos del rate limit /api/auth/* (20 r/m) para una operación que debería ser una.	`backend/src/main/java/com/redsocial/auth/AuthResource.java:33-40`, `AuthService.java:32-49`, `frontend/src/services/users.ts:7-22`.
H-19	M5	Alta	El draft de PostComposer se pierde al refrescar la pestaña, navegar a otra ruta o cambiar de cuenta. Si el usuario escribió 4000 caracteres y se distrajo, no hay sessionStorage que lo respalde.	`frontend/src/components/PostComposer.tsx` (sin useEffect de persistencia).
H-20	M5	Media	Branding inconsistente: la UI dice "ZENIT", pero `sw.js` usa el cache nodouni-push-v1 y el endpoint /__nodouni_push_state__. Evidencia de un refactor de nombre incompleto.	`frontend/public/sw.js` y `frontend/src/services/push.ts:6` (const CACHE = "nodouni-push-v1").
H-21	M5	Media	ChatDock fuerza la navegación a /chat cuando el viewport cruza el breakpoint mobile con el dock abierto. El usuario que estaba escribiendo pierde foco y scroll.	`frontend/src/components/ChatDock.tsx` (useMatchMedia + navigate).
H-22	M5	Media	CommentThread no tiene null-check sobre comment.autor.username. Si la API devuelve un comment huérfano (autor borrado), el JSX explota.	`frontend/src/components/CommentThread.tsx`.
H-23	M5	Baja	ConversationList muestra como preview el último mensaje sin diferenciar "mío" de "del otro". Si tu último mensaje es el visible, parece que el otro te contesta con tu propio texto.	`frontend/src/components/ConversationList.tsx` (messages.at(-1)?.texto).

[redacted]	Misión	Severidad	Hallazgo (resumen)	Evidencia clave (cont.)
H-24	M6	Media	El push silencioso multi-cuenta: el Service Worker mantiene un único owner en cache. Si te logueás como B sin desuscribir A, las notis de A llegan al endpoint pero el SW las descarta en silencio.	`frontend/public/sw.js` y `services/push.ts:55-89` (disablePush).
H-25	M6	Media	Sin test de JWT expirado: el test verifica exp - iat = 86400 pero nadie prueba el camino "token vencido → 401 → UX de cierre de sesión".	`backend/src/test/java/com/redsocial/auth/AuthResourceTest.java` (no hay método expired).
H-26	M6	Media	Sin test de race condition en POST /auth/registro concurrente. Dos requests simultáneos con el mismo username/email podrían crear un nodo duplicado si Neo4j no resolviera la constraint a tiempo.	`AuthResourceTest.java` y `ChatResourceTest.java` (este último sí cubre el caso de creación de conversación).
H-27	M6	Baja	RedisChatBroker reintenta con un timer fijo de 1 s; en producción con jitter alto podría causar tormenta de reconexiones. Aceptable para 2 réplicas, sería problema a 20+.	`backend/src/main/java/com/redsocial/chat/RedisChatBroker.java:80-86`.
H-28	M4	Alta	Credencial inmutable: sin cambio/recuperación de contraseña ni revocación de sesión JWT. Contraseña permanente ante filtraciones.	`backend/src/main/java/com/redsocial/auth/AuthResource.java:23,36,48`

2 Misión 1 — FROM ZERO: Reproducibilidad

Resultado: **Positivo** Reproducible. El primer intento con git clone + docker compose up -d –build levanta el entorno completo. Después del primer arranque (lento), arranques subsecuentes son segundos.

2.1 1.1 Resumen ejecutivo

El README.md está entre los mejores que he visto para un proyecto de este tamaño: explica arquitectura, requisitos, comandos, variables de entorno, modelo de grafo, endpoints, decisiones técnicas, respaldo/restauración y pruebas. El equipo hizo el trabajo de documentación.

2.2 1.2 Pasos realizados

Lectura completa del README.md (29.7 KB).

Inspección de `docker-compose.yml` (6.3 KB), `docker-compose.dev.yml`, Dockerfile de backend y frontend.

Verificación de los healthcheck de cada servicio.

Inspección del script keys-init que genera claves JWT y VAPID al primer arranque.

2.3 1.3 Hallazgos

H-01 · Media El primer arranque tarda porque MinIO se compila desde código fuente

Evidencia: `docker-compose.yml` → minio: { build: { context: ./infra/minio, target: minio } }. Es la imagen base, no se usa minio/minio: latest. Impacto: El primer up –build tarda varios minutos (descargar imágenes Maven, compilar backend, compilar MinIO). El README lo menciona pero no cuantifica. Recomendación: Documentar tiempo esperado (~10 min en hardware modesto) o cambiar a minio/minio: latest si la compilación no es necesaria.

H-02 · Positivo Si se borra el volumen app_keys, todos los JWT y suscripciones push caducan

Evidencia: `docker-compose.yml` job keys-init (líneas 73-103): genera el par RSA 2048 + VAPID solo si no existe. chmod 644 sobre las claves (ver H-10 en Misión 4). Impacto: Operacional: un operador que corre docker compose down -v por error invalida todos los tokens y suscripciones. El README lo documenta pero el sistema no avisa. Recomendación: Agregar un script `rotate-keys.sh` que migre tokens a una nueva clave pública sin forzar re-login (rotación de claves con período de gracia).

2.4 1.4 Conclusión de la misión

Positivo Reproducible. La promesa del README se cumple. El único punto de fricción real es la paciencia en el primer arranque.

3 Misión 2 — REQUIREMENTS HUNT

Resultado: Positivo Cumple, con gaps UX. Las 8 consultas Cypher (C1–C8) están implementadas y se verifican automáticamente al final del seed.

3.1 2.1 Resumen ejecutivo

El enunciado del trabajo pedía ”funcionalidades esenciales de una red social”. El sistema implementa:

- Registro e inicio de sesión con JWT
- Perfiles editables
- Seguimiento entre usuarios
- Publicaciones con imagen
- Feed personalizado
- Reacciones (1 tipo: LIKE)
- Comentarios con hilos de hasta 50 niveles (recortado a 200 más recientes)
- Recomendaciones (amigos de amigos)
- Chat en tiempo real (WebSocket + Redis Pub/Sub)
- Notificaciones push (Web Push + VAPID)
- Alcance de red y grados de separación
- Descubrimiento de publicaciones de tu red

3.2 2.2 Tabla de cumplimiento

3.3 2.3 Verificación automática del seed

El script `scripts/seed-demo.mjs` ejecuta, después de sembrar, una verificación que cubre C1, C2, C3, C4, C5 (con camino y sin camino), C7 y C8. Si alguna falla, el seed aborta con un mensaje claro. Esto es excelente porque evita el clásico ”subió el código pero los datos no se sembraron como dijo el README”.

Evidencia: `seed-demo.mjs:174-195` (async function verify), con checks nombrados C1 feed de ana, C2 sugerencias de ana, C3 en común ana/fabian, C4 alcance de ana, C5 separación ana/irene, C5 sin camino ana/julian, C7 descubrir de ana, C8 hilo del post de carla.

Requisito	Estado	Evidencia
Registro con validación	Cumple	`AuthResource.java:33-40`, validación Bean Validation con mensajes en español.
Login con JWT	Cumple	`AuthService.java:51-58`, JWT de 24h, RSA 2048.
Perfil editable (nombre, bio)	Cumple	`UserResource.java`, Profile record.
Búsqueda de usuarios	Cumple	users.search: '/usuarios?q='.
Seguir / dejar de seguir	Cumple	FollowResource, FollowService.
Crear publicación con imagen	Cumple	PostResource + MultipartForm, ImageValidator (PNG/JPEG/GIF, 5 MiB, 20 MP).
Feed personalizado (C1)	Cumple	FeedResource, FeedRepository con la consulta C1.
Reacciones (LIKE)	Cumple	PostService.react, una por usuario y post (MERGE).
Comentarios con hilos (C8)	Cumple	CommentRepository con RESPONDE_A recursivo, max 50 niveles, 200 comentarios por post.
Recomendaciones (C2)	Cumple	DiscoverResource/sugerencias, "amigos de amigos" con arranque en frío.
Chat uno-a-uno en tiempo real	Cumple	ChatSocket (WebSocket) + RedisChatBroker (Pub/Sub) + ChatRepository (Neo4j).
Notificaciones Web Push	Cumple	PostNotifier (@ObservesAsync PostCreated) + WebPushSender + SW en frontend.
Resistencia a fallos de Redis	Parcial	Si Redis cae, el mensaje se persiste en Neo4j y se entrega local, pero el otro participante no lo recibe en vivo hasta recargar. Documentado.
Imágenes en formatos modernos (WebP/HEIC)	No cumple	ImageValidator:22-25 solo acepta PNG/JPEG/GIF.
Bandeja de notificaciones in-app	No cumple	El push se muestra como notificación del SO, pero no hay vista en la SPA.
Cambio de contraseña	No cumple	No hay endpoint PUT /api/auth/password ni flujo de recuperación.
Bloqueo de usuarios	No cumple	No implementado.
Mensajes privados con archivos	No cumple	El chat es solo texto.

Cuadro 3: Tabla de Cumplimiento de Requisitos

3.4 2.4 Hallazgos

H-03 · El feed puede reportar más comentarios que los visibles en el hilo

Evidencia: `CommentRepository.java:23` MAX_THREAD_COMMENTS = 200. La consulta C1 reporta COUNT { (p)<-[: EN]-(: Comentario)<-[: RESPONDE_A*0..50]-(: Comentario) }, que cuenta el total. Si un post tiene 350 comentarios, el feed dirá "350" pero la vista de detalle solo traerá 200.

Impacto: Confusión menor. El frontend lo señala correctamente con la regla "Si el padre de una respuesta quedó fuera del recorte, el frontend la presenta como comentario principal para no ocultarla".

Recomendación: Documentar en el README o devolver un flag truncated: true en la respuesta de comentarios.

H-04 · Media No hay bandeja de notificaciones

Evidencia: `frontend/src/pages/SettingsPage.tsx` (sin sección de bandeja), `frontend/public/sw.js` solo dispara showNotification. Impacto: Si el usuario descarta la notificación nativa, no tiene forma de revisar qué se perdió. Común en redes sociales modernas. Recomendación: Agregar GET /api/notificaciones (eventos de posts de seguidos, reacciones a tus posts, comentarios a tus posts) y vista en la SPA. Es una buena Misión 7.

3.5 2.5 Conclusión de la misión

Positivo Cumple con los requisitos del enunciado. Los gaps identificados (bandeja, formatos de imagen modernos, cambio de contraseña) son ampliaciones, no incumplimientos.

4 Misión 3 — CODE AUTOPSY

Resultado: Media Aprobado con reservas. El backend tiene buena estructura por capas y un manejo honesto de eventos asíncronos. Los puntos a revisar son: el tamaño de PostService, el acoplamiento de contenido entre PostRepository y CommentRepository, el field injection en ChatSocket, y el spanglish estructural.

4.1 3.1 Resumen ejecutivo

El backend es Quarkus 3.33 LTS + Java 21. La organización de paquetes refleja features (auth, chat, comentarios, feed, media, notificaciones, posts, shared, social, usuarios) y la convención es Resource → Service → Repository → Driver. La mayoría de las clases ya migraron a constructor injection; quedan dos casos a actualizar.

4.2 3.2 Acoplamiento sano (lo que está bien)

Eventos asíncronos: PostService.create() emite PostCreated con events.fireAsync y no espera al push. Si mañana sacás las notificaciones, POST /api/posts ni se entera. Es el ejemplo de bajo acoplamiento del proyecto. `PostService.java:71-77`.

Inyección por constructor: Todos los Resources y Services del backend usan constructor injection. Ejemplos: AuthResource, AuthService, ChatService, PostService, CommentService, PushSubscriptionService. El contenedor CDI es explícito.

Interfaces para infraestructura: MediaStorage es una interfaz; PostService solo conoce MediaStorage, no S3MediaStorage. Si mañana cambias a Azure Blob, no tocás PostService. `MediaStorage.java:6-10`.

Dedupe en ChatSessions: LinkedHashMap con putIfAbsent y eviction al llegar a 4096 entries. Garantiza que un mensaje entregado por Redis y por la propia suscripción local no se envíe dos veces. `ChatSessions.java:31-37`.

Seed auto-verificable: `seed-demo.mjs:174-195` corre las 8 consultas Cypher al final y aborta si alguna falla. Es el equivalente a "tests de integración del happy path".

4.3 3.3 Acoplamiento que va a doler

H-05 · Alta PostService con 5 dependencias y método orquestador grande

Evidencia: `backend/src/main/java/com/redsocial/posts/PostService.java:21-26`:

```
public PostService(PostRepository repository, MediaStorage storage,
                   ImageValidator images, Event<PostCreated> events,
                   PostAssembler assembler) { ... }
```

El método create() (líneas 38-83) valida texto, valida imagen, sube a storage, escribe en Neo4j, dispara evento, ensambla respuesta. 46 líneas con cuatro try/catch para rollback.

Impacto: El blast radius es enorme. Si cambia la subida de imágenes, tocás el mismo método que crea posts. Si cambia la serialización del evento, también. Si cambia el contrato de Neo4j, también.

Recomendación: Partirlo:

```
class PostCreator {
    private final TextValidator text;
    private final PostImageAttacher images;
    private final PostPersister persister;
    private final PostEventPublisher events;
    PostResponse create(CreatePostCommand cmd) {
        text.validate(cmd.text());
        var stored = persister.commit(cmd, images.attach(cmd.file()));
        events.publish(stored);
        return assembler.response(stored);
    }
}
```

PostImageAttacher orquesta ImageValidator + MediaStorage + rollback. PostPersister se reduce a una transacción Neo4j. El método create queda en 8 líneas.

H-06 · Media PostRepository.PUBLIC_FIELDS consume CommentRepository.COUNT_OF_POST

Evidencia: `PostRepository.java:23-29`:

```
public static final String PUBLIC_FIELDS = """
    ...
    %s AS comentarios""".formatted(CommentRepository.COUNT_OF_POST);
```

Acoplamiento de contenido: una constante de otro paquete aparece en el SQL formateado. Si mañana cambias cómo se cuentan los comentarios (sumar respuestas indirectas, por ejemplo), recompilás posts aunque no hayas tocado su lógica.

Impacto: Bajo en el día a día (la query no cambia seguido), alto cuando toque migrar. Es una static final compartida que rompe el principio "cada paquete sabe de sí mismo".

Recomendación: Duplicar el string en PostRepository o mover COUNT_OF_POST a shared/cypher/. La duplicación es preferible al acoplamiento de imports cruzados.

H-07 · Media AuthService depende de UserService (servicio) en vez de UserRepository

Evidencia: `AuthService.java:23-26`, 32-49:

```
public AuthService(AuthRepository repository, UserService users) {
    this.repository = repository;
    this.users = users;
} // register() hace: return users.profile(id);
```

Acoplamiento transitivo: un cambio en UserService.profile() puede romper AuthService.register() sin tocar el paquete auth.

Impacto: Un UserService que hoy solo expone profile() mañana puede crecer y arrastrar dependencias que auth no necesita.

Recomendación: AuthService debería leer del UserRepository directamente (que es lo que hace para escribir). Si la lógica de profile() es no trivial, moverla a un ensamblador compartido o a un método del repositorio.

H-08 · Media ChatSocket con field injection

Evidencia: `ChatSocket.java:31-38`:

```
@Inject ChatService service;
@Inject ChatSessions sessions;
@Inject JsonWebToken jwt;
@Inject ObjectMapper json;
```

El resto del backend migró a constructor. ChatSocket quedó atrás.

Impacto: Inmutabilidad perdida (campos no final), dependencias implícitas (no se ven en el constructor), testabilidad peor (necesitás levantar CDI para mockear).

Recomendación: Migrar a constructor injection. Cambio mecánico, ~10 minutos.

H-09 · Positivo Spanglish estructural

Evidencia:

Paquetes en español: comentarios, notificaciones (`ClavePublica.java`, `SuscripcionRequest.java`, `Sugerencia.java`).

Records con campos en español: CommentRequest(texto, respondeA), CommentResponse(texto, fecha, autor, respuestas).

Métodos en inglés sobre esos mismos objetos: create(), thread(), save(), history().

Comentarios en inglés: // Shares the /posts root..., // Business error thrown....

Mensajes al usuario en español: mensaje no message.

Impacto: El costo cognitivo se duplica: para encontrar algo tenés que adivinar si es text o texto, parentId o respondeA. Además, el dominio "miente": DDD dice que el código tiene que hablar el idioma del negocio. Si tu negocio es una red social latina, respondeA está bien; entonces create debería ser crear.

Recomendación: Tres reglas documentadas en el README:

Código (identificadores + comentarios + commits): TODO en inglés. text, parentId, FollowerCount.

Lo que ve el usuario (mensajes de error, copy del frontend, descripciones OpenAPI): TODO en español. Ya lo hacés con mensaje y content/copy.

No migrar lo viejo de golpe. Congelá la regla para código nuevo desde hoy. Lo viejo se migra cuando toque cada módulo por otra razón.

4.4 3.4 Conclusión de la misión

Media Aprobado con reservas. El código tiene alma de un sistema distribuido bien pensado. Las reservas son:

Refactor de PostService en su primer sprint libre.

Migrar ChatSocket a constructor injection (cambio mecánico).

Eliminar el acoplamiento de contenido entre PostRepository y CommentRepository.

Documentar la regla de idioma antes de que entren más contribuidores.

5 Misión 4 — BREAK IT: Seguridad y pentesting controlado

Resultado: Alta Aprobado con reservas críticas. Se identifica una vulnerabilidad crítica (chmod 644 sobre privateKey.pem) que debe corregirse antes de cualquier despliegue fuera de localhost. El resto son endurecimientos progresivos.

5.1 4.1 Resumen ejecutivo

El sistema tiene un diseño de seguridad razonable: tokens JWT firmados con RSA, contraseña hasheada con bcrypt, push con VAPID, validaciones de entrada en cada endpoint, rate limiting en Nginx. Pero hay tres puntos rojos: la clave privada compartida con permisos demasiado abiertos, un timing attack detectable en el login, y la ausencia de Content-Security-Policy en el proxy.

5.2 4.2 Hipótesis → Prueba → Evidencia → Impacto

H-10 · Crítica CRÍTICA — Clave privada del JWT world-readable

Hipótesis: El volumen app_keys con chmod 644 permite que cualquier proceso en el host o contenedor con acceso al volumen firme tokens falsos.

Prueba: Lectura directa de `docker-compose.yml`:

```
keys-init:
  image: alpine:3.20
  volumes:
    - app_keys:/keys
  command: |
    chmod 644 /keys/*   # línea 102
```

Evidencia: `docker-compose.yml:102`. La clave privada queda con permisos rw-r–r– (644). Cualquier proceso con acceso de lectura al volumen puede usarla para firmar JWTs válidos.

Impacto: Crítico. Un atacante con acceso de lectura al host (compromiso de otro servicio, container escape, error de backup que termina en un bucket público) puede emitir tokens para cualquier subject y leer/escribir datos en nombre de cualquier usuario.

Recomendación:

Cambiar chmod 644 por chmod 600 en el job keys-init.

Cambiar el umask del proceso backend a 077 antes de leer.

Auditar quién tiene acceso al host y al volumen con docker volume inspect app_keys.

Documentar la rotación de claves (regenerar privateKey.pem invalida todos los JWTs).

H-11 · Alta Alta — User enumeration por timing en AuthService.login()

Hipótesis: Si el usuario no existe, el Optional se filtra sin correr bcrypt.matches. La rama ”no existe” termina antes que ”existe + contraseña mala”, y la diferencia es detectable con un número suficiente de muestras.

Prueba: Lectura directa de AuthService.login():

```
public LoginResponse login(LoginRequest request) {
    Credentials credentials = repository.findCredentials(request.username())
        .orElseThrow(InvalidCredentialsException::new);
    if (!BcryptUtil.matches(request.password(), credentials.passwordHash())) {
        throw new InvalidCredentialsException();
    }
    return new LoginResponse(jwtService.sign(credentials.id()), profile);
}
```

Evidencia: `AuthService.java:51-58` + `AuthRepository.java:51-66` (findCredentials retorna Optional<Credentials>). Si el Optional está vacío, BcryptUtil.matches no se ejecuta. Bcrypt con factor 10-12 toma ~100-300 ms; el retorno vacío toma ~5-20 ms (solo el round-trip a Neo4j).

Impacto: Un atacante puede enumerar usernames válidos midiendo el tiempo de respuesta, y luego enfocar el brute force en esos. El rate limit de Nginx (20 r/m) ayuda pero no cierra el canal.

Recomendación: Forzar siempre la corrida de bcrypt:

```
String hash = repository.findCredentials(request.username())
    .map(Credentials::passwordHash)
    .orElse(BCRYPT_OF_DUMMY);
```

El BCRYPT_OF_DUMMY se calcula una vez al arranque y se guarda. Así ambos caminos demoran lo mismo.

H-12 · Alta Alta — Sin Content-Security-Policy en Nginx

Hipótesis: Un XSS en el frontend podría cargar scripts arbitrarios desde cualquier origen, robando el JWT de localStorage.

Prueba: Lectura de `frontend/nginx.conf`:

```
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```

No hay Content-Security-Policy.

Evidencia: `frontend/nginx.conf` en el bloque server. Tres headers presentes, CSP ausente.

Impacto: Un comentario con un payload XSS (texto sin escape, markdown mal procesado, dangerouslySetInnerHTML) puede cargar un script externo que lea localStorage.getItem("nodouni.session") y exfiltre el JWT.

Recomendación: Agregar una CSP estricta. Como el frontend es Vite + Tailwind sin assets cross-origin:

```
add_header Content-Security-Policy "default-src 'self';
  script-src 'self'; style-src 'self' 'unsafe-inline';
  img-src 'self' data: blob:;
  connect-src 'self' wss: https://*.push.services.mozilla.com;" always;
```

Los endpoints push (Mozilla, FCM, Apple) requieren connect-src permisivo para esos orígenes específicos.

H-13 · Media JWT en localStorage (vulnerable a XSS)

Evidencia: `frontend/src/lib/session.ts:3` SESSION_KEY = "nodouni.session". `api.ts:26,31` lo lee y lo manda en cada request. Impacto: Si H-12 no se cierra, H-13 amplifica el daño. Aceptable con JWT de 24h y sin refresh, pero no guardar nada sensible además del token en localStorage. Recomendación: Documentar en el README. Considerar migrar a cookie httpOnly + SameSite=Lax con CSRF token cuando se implemente refresh.

H-14 · Media El frontend desloguea en cualquier 401 sin distinguir vencido vs inválido

Evidencia: `frontend/src/lib/api.ts:40-41`:

```
if (response.status === 401 && token && getSession()?.token === token) {
    setSession(null);
}
```

Impacto: Un 401 transitorio (skew de reloj, request con token viejo en otra pestaña) desloguea al usuario sin aviso. Con JWT de 24h y sin refresh, ese logout duele. Recomendación: Distinguir:

WWW-Authenticate: Bearer error="invalid_token" → token malformado o mal firmado → logout.

Sin error específico y tokenExpiresAt(token) <now + 30s → pre-expirado → intentar refresh (cuando exista).

Otros 401 → mostrar mensaje, no desloguear.

H-15 · Media ImageValidator no acepta WebP/HEIC

Evidencia: `backend/src/main/java/com/redsocial/media/ImageValidator.java:22-25`:

```
private static final Map<String, ImageType> TYPES = Map.of(
    "png",  new ImageType("image/png",  "png"),
    "jpeg", new ImageType("image/jpeg", "jpg"),
    "gif",  new ImageType("image/gif",  "gif")
);
```

Impacto: UX. Los celulares modernos sacan WebP/HEIC por defecto. El usuario tiene que convertir antes de subir. Recomendación: Agregar "webp" → image/webp y opcionalmente HEIC (con libheif). Requiere actualizar la imagen base del backend o agregar la dependencia nativa.

H-16 · Media Defaults que no pueden llegar a prod sin alerta

Evidencia: `docker-compose.yml` usa VAPID_SUBJECT: ${VAPID_SUBJECT:-mailto: admin@example.com} y NEO4J_PASSWORD: ${NEO4J_PASSWORD:-devpassword}. Impacto: Si el operador olvida cambiar el .env antes de desplegar, el sistema funciona pero con credenciales de dev. Recomendación: Agregar un check de arranque en Quarkus:

```
@ApplicationScoped
public class ProdSecretsCheck {
    void onStart(@Observes StartupEvent e) {
        if (ConfigSource.from("prod") &&
            (config.neo4jPassword().equals("devpassword") ||
             config.vapidSubject().contains("example.com"))) {
            throw new IllegalStateException("Dev defaults still set in prod");
        }
    }
}
```

H-17 · Media Sin observabilidad distribuida

Evidencia: No hay dependencia de quarkus-micrometer ni quarkus-opentelemetry en el pom.xml. No se propaga X-Request-Id en Nginx. Impacto: Con 2 réplicas + round robin, cuando un usuario reporta "me falló el post", no podés saber si fue backend-1 o backend-2 salvo por GET /info o el upstream_addr del log de Nginx. Recomendación:

Agregar quarkus-micrometer-registry-prometheus + endpoint /q/metrics.

En Nginx: proxy_set_header X-Request-Id $request_id; (usar uuid4 si $request_id no está habilitado).

En el backend: loggear requestId en cada línea.

Incluir app.instance-id en cada log (ya está en `application.properties`).

H-28 · Alta Credencial inmutable: sin cambio/recuperación de contraseña ni revocación

Hipótesis: La contraseña es el único factor de autenticación y es inmutable. Si se filtra, la víctima no puede cambiarla (no hay endpoint) ni expulsar al atacante (el JWT stateless dura 24 h y no se puede revocar).

Prueba: Inspección de `AuthResource.java:23,36,48` — @Path("/auth") solo expone /registro y /login; ninguna ruta de contraseña. `AuthService.java:51-64` — login() firma el JWT; no hay método de cambio de contraseña. `application.properties:23` — smallrye.jwt.new-token.lifespan = 86400 (24 h). `frontend/src/context/AuthProvider.tsx:68-71` — logout solo borra el estado local.

Evidencia: `backend/src/main/java/com/redsocial/auth/AuthResource.java:23,36,48`, `backend/src/main/java/com/redsocial/auth/AuthService.java:51-64`, `backend/src/main/resources/application.properties:23`, `frontend/src/context/AuthProvider.tsx:68-71`.

Impacto: Alta. El acceso del atacante es irrevocable hasta por 24 h por token emitido, y permanente mientras el atacante pueda volver a loguearse con la contraseña comprometida. "Cerrar sesión" en el cliente no invalida el token en el servidor.

Recomendación: Añadir tokenVersion en (: Usuario) como claim validado por petición contra el JWT, y exponer PUT /api/auth/password (solicitando la contraseña actual) que incremente este contador. Así, al cambiar la contraseña se invalidan instantáneamente todos los tokens previos en ambas réplicas.

5.3 4.3 Lo que SÍ está bien

Nginx limita correctamente: auth_zone: 20r/m burst=40 para /api/auth/*, write_zone: 120r/m burst=60 para métodos no-GET fuera de auth. Lecturas no consumen cuota.

El SW no loggea tokens: `frontend/public/sw.js` no guarda el JWT ni la query.

client_max_body_size 6m está bien calibrado para el límite de 5 MiB del ImageValidator.

Push endpoint URL validador: PushEndpointValidator rechaza local hostnames, IPv6, y exige HTTPS + dominio público.

ChatTokenAuthentication con cap de 16384 chars en el query param del WS.

Lock distribuido en suscripciones push: PushRepository.subscribe usa executeWrite con LOCK_USER (incrementa pushSubscriptionVersion) para serializar suscripciones concurrentes desde ambas réplicas.

Web Locks API en el frontend: `services/push.ts` usa navigator.locks.request("nodouni-push", ...) para evitar doble subscribe concurrente.

5.4 4.4 Conclusión de la misión

Alta Aprobado con reservas críticas. Corregir H-10 (chmod 600) antes de cualquier despliegue real es obligatorio. H-11, H-12 son endurecimientos importantes. El resto puede esperar al siguiente sprint.

6 Misión 5 — UX UNDER FIRE

Resultado: Media Aprobado con fricción. El sistema es aprendible y consistente, pero tiene fricciones de productividad (drafts muertos, registro de dos pasos, branding inconsistente).

6.1 5.1 Resumen ejecutivo

La SPA está hecha con React 19 + TypeScript + Vite + React Router. La accesibilidad está cubierta con skip-link, aria-label, aria-pressed, aria-current, role=dialog, prefers-color-scheme. El layout tiene dos variantes responsive (sidebar vs bottom nav). El copy está centralizado en `content/copy.ts` y similares.

6.2 5.2 Flujos evaluados

6.2.1 5.2.1 Registro

Pasos: Pantalla /registro → completar username, email, password, nombre → POST /auth/registro → redirige a /login.

Fricción H-18 (Alta): El registro es de dos round-trips. El backend devuelve Profile, no token. El usuario tiene que volver a tipear usuario y contraseña.

Evidencia: `frontend/src/services/users.ts:14-22` (register retorna Profile, no token). `AuthResource.java:33-40` (register retorna Profile). El README dice ”registro e inicio de sesión” como funcionalidades, no las fusiona.

Recomendación: Devolver LoginResponse con Profile embebido, o auto-loguear al usuario después de registrarse. Ambos lados son cambios chicos.

6.2.2 5.2.2 Login

Pasos: /login → usuario + contraseña → POST /auth/login → token guardado en localStorage → redirige a /feed.

Fricción menor: No hay ”olvidé mi contraseña” (ver H-04 equivalente en Misión 2).

Positivo: AuthProvider arma un timer con tokenExpiresAt(session.token) para auto-desloguear cuando vence. Pero ese timer se reinicia en cada useEffect, lo que significa que no hay chequeo proactivo del lado del cliente mientras hay actividad.

6.2.3 5.2.3 Crear publicación

Pasos: /feed → click ”Publicar” → PostComposer con textarea + upload de imagen → enviar.

Fricción H-19 (Alta): Si el usuario refresca la pestaña, navega a otra ruta, o cambia de cuenta, el texto y la imagen se pierden sin aviso. No hay sessionStorage que los respalde.

Evidencia: `frontend/src/components/PostComposer.tsx` no tiene un useEffect que persista. El subagente de revisión lo confirmó.

Fricción adicional: El contador de caracteres cuenta [...text.trim()].length, que son code points, no grapheme clusters. Un emoji con ZWJ (familia [emoji familia]) cuenta como varios y puede bloquear el envío legítimamente.

Recomendación: Persistencia debounced en sessionStorage (lifespan: la sesión), y conteo con Intl.Segmenter para graphemes.

6.2.4 5.2.4 Seguir a un usuario

Pasos: /explorar o /perfil → click ”Seguir” → POST /usuarios/{id}/seguir.

Positivo: Feedback inmediato (botón cambia a ”Dejar de seguir” + recuento actualizado).

Sin fricción notable.

6.2.5 5.2.5 Comentar

Pasos: Click en una publicación → CommentThread + CommentComposer → enviar.

Fricción H-22 (Media): Sin null-check sobre comment.autor.username. Si la API devuelve un comment huérfano (autor borrado), el JSX explota.

Fricción menor: El estado expanded/replyingTo es local al componente. Al cambiar de post y volver, los hilos se colapsan.

Positivo: El draft se preserva si falla el envío (setError(errorMessage(cause)) sin limpiar text).

6.2.6 5.2.6 Chat

Pasos: /chat o ChatDock flotante → seleccionar conversación → escribir → WS envía.

Positivo: ChatProvider remonta al cambiar de token (key={session?.token}), lo que blinda contra filtrado de sockets entre cuentas. Excelente.

Positivo: useChatConnection reconecta con backoff exponencial 1s → 30s.

Positivo: Drafts de ConversationPanel persisten por id de conversación en el ChatContext.

Fricción H-21 (Media): ChatDock fuerza la navegación a /chat cuando el viewport cruza el breakpoint mobile con el dock abierto. Pierde foco y scroll.

Fricción H-23 (Positivo): El preview del último mensaje en ConversationList no diferencia ”mío” vs ”del otro”.

6.2.7 5.2.7 Activar notificaciones

Pasos: /configuracion → click ”Activar notificaciones” → permiso del browser → suscripción push.

Fricción menor: Si el permiso está denied y no está habilitado, el botón queda disabled sin explicar cómo destrabarlo desde el browser.

Fricción menor: Si la suscripción falla, no hay reintento automático.

Positivo: Web Locks API previene doble subscribe concurrente.

Positivo: assertSession(userId, token) valida que la sesión no cambió durante la operación async.

6.3 5.3 Hallazgos específicos

H-20 · Media Branding inconsistente (ZENIT vs nodouni)

Evidencia:

`frontend/src/components/AuthPanel.tsx:5` aria-label=”ZENIT”.

`frontend/src/content/copy.ts:3` brand: ”ZENIT”.

`frontend/src/services/push.ts:6` const CACHE = ”nodouni-push-v1”.

`frontend/src/services/push.ts:7` const STATE = ”/__nodouni_push_state__”.

`frontend/public/sw.js` (mismo cache name y endpoint).

Impacto: Evidencia de un refactor de nombre incompleto. Si el equipo de soporte revisa logs del SW, ve ”nodouni” y se confunde.

Recomendación: Hacer un grep -r ”nodouni” frontend/src frontend/public y reemplazar.

H-22 · Media CommentThread sin null-check sobre el autor

Evidencia: `frontend/src/components/CommentThread.tsx` renderiza comment.autor.username directamente. Si la API devuelve un comment huérfano (autor borrado, FK rota en Neo4j), el render explota.

Impacto: Pantalla en blanco en el peor caso.

Recomendación: Validar comment.autor && comment.autor.username antes de renderizar, o agregar un <ErrorBoundary>específico para hilos de comentarios.

H-24 · Media Push silencioso multi-cuenta

Evidencia: `frontend/public/sw.js` mantiene UN SOLO owner en cache. `services/push.ts:55-89` (disablePush) verifica el owner pero solo si coincide con el usuario actual.

Impacto: Si el usuario A se desloguea y entra B sin desuscribir A, las notis de A siguen llegando al endpoint (Chrome las entrega), pero el SW las descarta en silencio. El usuario no sabe que está perdiendo notificaciones.

Recomendación: Al logout en AuthProvider, llamar disablePush automáticamente (de hecho, ya lo hace: `AuthProvider.tsx:30-37`). El gap es si la pestaña se cierra sin disparar el evento de logout.

6.4 5.4 Lo que SÍ está bien

Accesibilidad: skip-link, aria-label, aria-pressed, aria-current, role=dialog, prefers-color-scheme, focus management en "Mostrar más".

Manejo de errores: ApiError tipado, mensajes en español.

Remount por token: AppLayout y ChatProvider remontan al cambiar de token. Excelente defensa contra filtrado de estado entre cuentas.

i18n lista: los strings de UI están en `content/copy.ts`, no hardcodeados en componentes.

Responsive: sidebar en desktop, bottom nav en mobile, dock adaptable.

6.5 5.5 Conclusión de la misión

Media Aprobado con fricción. El sistema es coherente y aprendible, pero la fricción real está en los flujos de productividad (drafts, registro, push multi-cuenta). Las correcciones son de baja complejidad.

7 Misión 6 — CHAOS MODE

Resultado: Positivo Buena resiliencia, con gaps de testing. El sistema tiene retry exponencial en WS, retry con backoff en REST, dedupe en ChatSessions, y reconexión en RedisChatBroker. Pero faltan tests de los caminos de fallo más importantes (token JWT expirado, race en registro, reconexión WS con mensajes en vuelo).

7.1 6.1 Resumen ejecutivo

La resiliencia se construyó por dos lados: el backend con reconexión automática (Redis broker) y el frontend con reintentos y backoff (WS, history, api en general). El test suite es sólido en chat y débil en auth/posts.

7.2 6.2 Modos de fallo analizados

7.2.1 6.2.1 Neo4j se detiene temporalmente

Comportamiento esperado: El backend no puede escribir ni leer. Los requests devuelven 500 ERROR_INTERNO (vía ExceptionMappers.unexpected).

Comportamiento real: ExceptionMappers.unexpected loguea el error y devuelve ERROR_INTERNO. El frontend reintenta history 3 veces (H-XX).

Recuperación: Cuando Neo4j vuelve, el frontend no necesita refrescar manualmente: el próximo request funciona. Los health checks de Docker reinician automáticamente el contenedor.

Gap: No hay circuit breaker. Si Neo4j se demora 30s, cada request espera 30s antes de fallar. Con 2 réplicas, el LB no las saca de rotación.

7.2.2 6.2.2 Redis se detiene

Comportamiento esperado: El chat no se entrega entre réplicas. El historial queda en Neo4j.

Comportamiento real (backend): ChatService.send() intenta `broker.publish(saved).await().atMost(Duration.ofSeconds(2))` y si falla, loguea warnf("Redis unavailable..."). Después llama sessions.deliver(saved) local, así que el emisor siempre ve su mensaje.

Recuperación: RedisChatBroker (`RedisChatBroker.java:80-86`) reconecta con timer de 1s al detectar disconnected(). Backoff lineal, no exponencial.

Gap: El receptor en otra réplica no ve el mensaje en vivo. Solo lo ve al recargar historial por REST. Esto es un tradeoff conocido del diseño, documentado en el README, pero el usuario lo vive como bug.

7.2.3 6.2.3 Backend-1 se reinicia

Comportamiento esperado: Los sockets conectados a backend-1 se cierran. El frontend reconecta.

Comportamiento real: useChatConnection tiene backoff 1s → 2s → 4s → ... → 30s. Al reconectar, el handshake con ?token= re-valida el JWT. Si el token venció, el backend cierra con 1008 Usuario no disponible o 1008 JWT vencido.

Recuperación de mensajes perdidos: reconnect() en ChatProvider recarga chat.list y vuelve a llamar loadHistory(id) para cada conversación cargada. Excelente.

Gap: Los mensajes enviados durante el reinicio se pierden del feed en vivo (el send() en el frontend retorna false si el socket no está OPEN). No hay cola de outbox en el cliente.

7.2.4 6.2.4 Neo4j se corrompe (constraint violation masiva)

Comportamiento: Las constraints están como IF NOT EXISTS (ver `Neo4jConstraints.java:18-25`). Si se reinicia Neo4j con una versión anterior del schema, las constraints se vuelven a crear. Bien.

Gap: Si dos requests concurrentes intentan crear el mismo username, el primero gana y el segundo recibe Neo.ClientError.Schema.ConstraintViolationFailed. El AuthService lo captura y lo traduce a 409 USERNAME_EN_USO. Pero hay una ventana mínima donde ambos pueden leer "no existe" antes de que el constraint valide.

7.2.5 6.2.5 Token JWT expira

Comportamiento backend: El endpoint protegido responde 401 vía ExceptionMappers.authenticationFailed. El header WWW-Authenticate: Bearer está presente.

Comportamiento frontend: `api.ts:40-41` desloguea en cualquier 401. El AuthProvider arma un timer de tokenExpiresAt que se ejecuta cuando vence, llamando setSession(null). El doble deslogueo es idempotente, así que no hay problema funcional, pero la UX es brusca.

Gap de test (H-25): No hay un test que pruebe "token vencido → 401 → UX de cierre de sesión". El test verifica la duración (exp - iat = 86400) pero no el camino de fallo.

7.2.6 6.2.6 Acciones concurrentes (reaccionar dos veces al mismo post rápido)

Comportamiento: PostService.react usa MERGE en Cypher con ON CREATE SET. La segunda llamada no duplica la relación, solo actualiza el timestamp. Idempotente.

Comportamiento frontend: PostCard.toggleReaction tiene if (reactionBusy) return y setReactionBusy(true). Seguro.

7.2.7 6.2.7 Frontend con un servicio no disponible (Neo4j caído)

El frontend hace await fetch("/api/..."). Si el backend está arriba pero Neo4j caído, recibe 500. `api.ts:30` lo traduce a ApiError(0, { error: "CONEXION", mensaje: copy.connectionError }). Falla genérica, no específica.

El feed no se distingue entre "Neo4j caído" y "backend caído". El usuario ve "Error de conexión" sin contexto.

7.3 6.3 Hallazgos

H-25 · Media No hay test de JWT expirado

Evidencia: `backend/src/test/java/com/redsocial/auth/AuthResourceTest.java` verifica exp - iat = 86400 pero no prueba el flujo "vencido → 401 → UX". Impacto: El camino más probable a 24h vista no está cubierto. Recomendación: Test con un JWT firmado con lifespan=1s + Thread.sleep(1500) + request → assert 401.

H-26 · Media No hay test de race en POST /auth/registro

Evidencia: `AuthResourceTest.java` cubre duplicados secuenciales (409), pero no concurrentes. Impacto: Ventana mínima donde dos requests podrían pasar la lectura "no existe" antes de la escritura constraint. Depende del aislamiento de Neo4j. Recomendación: Test con ExecutorService lanzando 4 registros en paralelo con el mismo username → assert que solo 1 devuelve 201 y 3 devuelven 409.

H-27 · Positivo RedisChatBroker reconecta con backoff lineal fijo

Evidencia: `RedisChatBroker.java:80-86` vertx.setTimer(1000, ...). Sin jitter, sin backoff exponencial. Impacto: Con 2 réplicas no hay problema. Con 20+ en una tormenta de red, todas las reconexiones golpean Redis al mismo tiempo. Aceptable hoy. Recomendación: Cambiar a min(1000 * 2^n + jitter, 30000).

7.4 6.4 Lo que SÍ está bien

RedisChatBroker tiene reconexión robusta: la lógica disconnected() → setTimer(1000, connect) se ejecuta dentro de synchronized y verifica stopped para no fugar timers. Es código defensivo serio.

useChatConnection con backoff exponencial: 1s → 2s → 4s → ... → 30s. Es la práctica recomendada para clientes WebSocket.

chat.history reintenta solo lecturas idempotentes: 3 intentos con delay incremental (1s, 2s), solo en errores de red (0, 502, 503, 504). Nunca reintenta un POST, lo que evita duplicar posts o comentarios.

Tests de chat son sólidos: `ChatSocketTest.java` cubre tokens expirados, tokens con issuer incorrecto, token doble (?token=x&token=y), conexión concurrente, fan-out a múltiples sesiones, failedBroker (mensaje se persiste localmente aunque Redis esté caído). El test suite más maduro del proyecto.

Tests de chat cubren idempotencia concurrente: `ChatResourceTest.createIsIdempotentInBothDirectionsEvenConcurrently` lanza 6 requests en paralelo y verifica una sola conversación.

Nginx proxy_next_upstream con cap de 2 tries y timeout 2s/5s: si una réplica está caída, el LB no se queda esperando.

Seed auto-verificable: aborta el seed si alguna de las 8 consultas falla.

7.5 6.5 Conclusión de la misión

Positivo Buena resiliencia, con gaps de testing. El sistema degrada con elegancia (mensajes en historial, reconexión automática, dedupe). Faltan tests de los caminos de fallo más comunes (JWT expirado, race en registro), pero la implementación parece sólida.

Anexo A — Hallazgos del equipo original (mapeados a las misiones)

Esta sección preserva y referencia los hallazgos de la auditoría manual que el equipo entregó, para que el documento final sea autocontenido. Cada hallazgo está mapeado a la misión correspondiente.

A.1 Token, sesión y autenticación (M4, M5)

"Es un solo token, sin refresh. `LoginResponse.java` es un record con un solo campo token, y AuthService.login() lo genera con `Jwt.subject(id).upn(username).sign()`. No hay ningún RefreshToken en el grafo — busqué y no existe. El backend mismo lo dice en `AuthResource.java:20-21`: 'el login emite el JWT que todo endpoint privado requiere; el servidor no guarda sesión'. Es stateless puro, con vida útil de 24h."

Confirmado. `AuthService.java:51-58` genera el JWT con `Jwt.subject(credentials.id()).upn(credentials.username()).sign()`. `application.properties` define `smallrye.jwt.new-token.lifespan=${JWT_LIFESPAN_SECONDS:86400}` (24h).

"Y sí, vive en localStorage. `frontend/src/lib/session.ts:3` define SESSION_KEY = "nodouni.session" y guarda el JSON { token, user } ahí. Después `frontend/src/lib/api.ts:26,31` lo lee con getSession()?.token y lo manda en cada request como Authorization: Bearer <token>. Si el backend devuelve 401, `api.ts:40-41` hace setSession(null) — o sea, te desloguea."

Confirmado y elevado a H-13 (Media , M4) y H-14 (Media , M5).

A.2 Acoplamiento sano y malsano (M3)

"Lo sano: cada feature tiene Resource → Service → Repository con inyección por constructor. `PostNotifier.java:36` escucha PostCreated con @ObservesAsync. PostService dispara el evento con fireAsync y NO espera el push. MediaStorage es interfaz."

Confirmado y referenciado en la sección 3.2 de M3.

"Lo acoplado de más: `PostService.java:22-26` tiene 5 dependencias. AuthService → UserService → UserRepository. PostRepository.PUBLIC_FIELDS:23-29 importa CommentRepository.COUNT_OF_POST. Hotspots con fan-in altísimo: ExceptionMappers.api (fan-in 52), PostRepository.react (51), `lib/api.ts`: api (24)."

Confirmado y elevado a H-05 (Alta), H-06 (Media), H-07 (Media) en M3.

A.3 Field injection en ChatSocket (M3)

"Esos 4 @Inject en campos tendrían que ser constructor injection con final."

Confirmado y elevado a H-08 (Media) en M3.

A.4 Spanglish estructural (M3)

"Paquetes en español: comentarios, notificaciones (`ClavePublica.java`, `SuscripcionRequest.java`, `Sugerencia.java`). Records con campos en español: CommentRequest(texto, respondeA). Métodos en inglés sobre esos mismos objetos: create(), thread(), save(), history(). Mensajes al usuario en español: 'El usuario no existe'."

Confirmado y elevado a H-09 (Positivo) en M3, con la recomendación concreta de tres reglas documentadas.

A.5 WebSocket pegajoso vs REST libre (M3, M6)

"Un WS conectado NO vuelve a pasar por el balanceador. El round robin elige UNA sola vez: en el handshake. El problema real es el OTRO caso: tu amigo manda un mensaje y su request REST cae en backend-2. ¿Cómo te llega? Por Redis. ChatService.send() hace las DOS cosas: broker.publish(saved) al canal Redis + sessions.deliver(saved) local. Cada réplica está suscripta."

Confirmado. ChatService.send() (líneas 60-72): hace repository.save() → broker.publish() → sessions.deliver(). RedisChatBroker suscribe commands.subscribe(CHANNEL, sessions:: deliver, this:: disconnected, ...). Excelente diseño.

A.6 Escalabilidad: chat dedicado o proxy (M3, M6)

"Si el chat escala distinto que el REST, SÍ, backend dedicado. Hoy tus 2 réplicas son monolitos clonados. Si el problema es 'nginx dentro del frontend me hace ruido', SÍ, proxy dedicado (Traefik/Caddy). Lo que NO haría: ip_hash."

Recomendación incorporada a la sección 6.2.1 y 6.2.2 de M6.

A.7 Permisos de clave privada (M4, Crítica)

”La privada del JWT queda legible por todos — No cumple lo más grave. `docker-compose.yml:102` hace chmod 644 /keys/* y monta app_keys: /keys: ro en AMBOS backends. Tiene que ser 600 (o 640 con grupo), nunca 644.”

Confirmado y elevado a H-10 (Crítica CRÍTICA) en M4.

A.8 Registro sin logueo (M5)

”El registro no loguea — Alta fricción UX. AuthService.register():32-49 devuelve Profile, no token. El frontend (users.register → api) obliga a registrarse Y después loguearse.”

Confirmado y elevado a H-18 (Alta) en M5.

A.9 401 desloguea sin piedad (M4, M5)

”El 401 te patea la sesión sin piedad — Alta . `api.ts:40-41`: cualquier 401 con el token actual hace setSession(null). Distinguí expirado vs inválido antes de borrar.”

Confirmado y elevado a H-14 (Media) en M4.

A.10 Imágenes sin WebP/AVIF (M5)

”Imágenes sin WebP/AVIF — Alta gap de producto. ImageValidator:22-25 solo acepta png/jpeg/gif (5 MiB, 20 MP). Los celulares modernos sacan WebP/HEIC por defecto.”

Confirmado y elevado a H-15 (Media) en M4.

A.11 Redis 100 % efímero (M6)

”Redis 100 % efímero — Cumple bien, pero asumí la ventana. redis: –save ”” –appendonly no + tmpfs: correcto para pub/sub, pero si Redis cae justo en el publish, el mensaje queda en Neo4j y solo se entrega local. El receptor lo ve al recargar historial, no en vivo.”

Confirmado. Documentado en la sección 6.2.2 de M6.

A.12 Defaults que no pueden llegar a prod (M4)

”VAPID_SUBJECT=mailto: admin@example.com y NEO4J_PASSWORD=devpassword funcionan en dev, pero nada te frena si se te olvida cambiarlos.”

Confirmado y elevado a H-16 (Media) en M4.

A.13 Sin observabilidad distribuida (M4)

”Sin observabilidad distribuida — Alta . Tenés /q/health y INSTANCE_ID como env, pero no veo Micrometer/OpenTelemetry ni correlation-ID en los logs.”

Confirmado y elevado a H-17 (Media) en M4.

Anexo B — Hallazgos propios que el equipo original no tenía

Estos hallazgos surgieron de la revisión cruzada (auditoría propia) y no estaban en la auditoría manual del equipo. Es la ”otra auditoría” que se pidió.

B.1 (M1) `seed-demo.mjs` exige base vacía y aborta con mensaje claro

Origen: Revisión del seed. Detalle: Si el usuario ya existe, el seed aborta con El usuario ”X” ya existe. El script parte de una base vacía: ejecutar ”docker compose down -v” y volver a levantar el stack. Veredicto: Positivo. Patrón ”fail loud” que evita datos corruptos. Referencia de buena práctica para otros scripts del proyecto.

B.2 (M3) ChatSessions.deliver tiene dedupe con eviction bounded

Origen: Revisión de `ChatSessions.java`. Detalle: LinkedHashMap con putIfAbsent y eviction al llegar a 4096. Esto previene que un mensaje entregado por Redis y por la propia suscripción local se envíe dos veces. El 4096 es un cap razonable (es ~16 KB de IDs, no es un memory leak). Veredicto: Patrón a copiar en otros lugares con fan-out (notificaciones push, por ejemplo).

B.3 (M3) PushRepository.subscribe usa lock distribuido en Neo4j

Origen: Revisión de `PushRepository.java:22-26`. Detalle: LOCK_USER hace SET u.pushSubscriptionVersion = coalesce(u.pushSubscriptionVersion, 0) + 1. Esto serializa suscripciones concurrentes desde ambas réplicas del backend, evitando race entre subscribe y unsubscribe. Veredicto: Patrón avanzado, raro de ver en proyectos académicos. Digno de mención en la defensa del proyecto.

B.4 (M4) El SW no loggea tokens (positiva)

Origen: Revisión de `frontend/public/sw.js`. Detalle: El Service Worker no escribe el JWT ni la query en ningún cache. Solo guarda metadata del owner (userId + endpoint) y la suscripción push. Si el operador revisa los logs del SW, no ve tokens. Veredicto: Alineado con H-17 (observabilidad) pero a nivel local. Bien.

B.5 (M5) useRemote hook con cancelaciones consistentes

Origen: Revisión de `ChatProvider.tsx:18-23`, 49-95. Detalle: Cada loadHistory(id) arma un AbortController y aborta el anterior con requests.current.get(id)?.abort(). Esto previene race conditions cuando el usuario cambia de conversación rápido. Veredicto: Patrón defensivo bien aplicado. El useRemote abstrae esto y lo aplica consistentemente.

B.6 (M5) AppLayout remonta ChatProvider con key={session?.token}

Origen: Revisión de `AppLayout.tsx`. Detalle: Cuando el usuario cambia (logout + login con otro token), el ChatProvider se desmonta y se vuelve a montar. Esto blinda contra filtrado de sockets, drafts o estados entre cuentas. Veredicto: Defensa en profundidad notable. No es común ver este nivel de cuidado en proyectos académicos.

B.7 (M6) `seed-demo.mjs` corre 8 verificaciones Cypher al final

Origen: Revisión del seed (ver B.1). Detalle: Después de sembrar, el script ejecuta feed, suggestions, mutuals, reach, separation (con y sin camino), discover, thread y aborta si alguna falla. Esto es esencialmente un test de integración del happy path. Veredicto: Patrón a copiar para futuras features: "si el seed se completa pero las queries fallan, abortar con mensaje claro".

B.8 (M6) Tests de chat cubren casos no triviales

Origen: Revisión de `ChatSocketTest.java` y `ChatResourceTest.java`. Detalle:

`persistsBeforePublishingAndDeliversOnceToAllParticipantSessionsOnly`: verifica que el mensaje se persiste ANTES de publicarse a Redis, y que solo se entrega una vez aunque haya múltiples sesiones del mismo usuario.

createIsIdempotentInBothDirectionsEvenConcurrently: 6 requests en paralelo creando la misma conversación, verifica una sola.

`cursorPaginatesTiesWithoutSkippingOrRepeatingAfterANewMessage`: previene el bug clásico de skip/repeat.

failedBroker: mensaje se persiste localmente aunque Redis esté caído.

Veredicto: El test suite más maduro del proyecto. Es una referencia para escribir los tests faltantes en auth y posts (H-25, H-26).

Anexo C — Resumen de la auditoría manual del equipo

Una vista compacta de los hallazgos de la auditoría manual del equipo, ordenadas por severidad. No incluye los hallazgos nuevos de esta auditoría cruzada (que están en Anexo B y en la tabla del Resumen Ejecutivo).

Hallazgo (resumen del equipo)	Severidad	Misión	Elevado a
Token único sin refresh, 24h, en localStorage	Media Media	M4, M5	H-13, H-14
Acoplamiento sano: eventos async, interfaces	Positivo Positivo	M3	Referencia en § 3.2
PostService con 5 deps (god-service incipiente)	Alta Alta	M3	H-05
AuthService → UserService → UserRepository	Media Media	M3	H-07
PostRepository importa constante de CommentRepository	Media Media	M3	H-06
Fan-in altísimo en ExceptionMappers/PostRepository/lib/api	Positivo Positivo (a manejar)	M3	§ 3.2
ChatSocket con field injection	Media Media	M3	H-08
Spanglish estructural	Positivo Baja	M3	H-09
Alternativas a monolito: chat dedicado, proxy separado	Info Info	M3, M6	§ 6.2.1, § 6.2.2
privateKey.pem chmod 644 (world-readable)	Crítica Crítica	M4	H-10
Registro devuelve Profile, no token (2 round-trips)	Alta Alta	M5	H-18
401 desloguea sin distinguir vencido vs inválido	Media Media	M4, M5	H-14
Imágenes sin WebP/AVIF	Media Media	M4, M5	H-15
Redis 100 % efímero (documentar tradeoff)	Positivo Positivo	M6	§ 6.2.2
VAPID_SUBJECT y NEO4J_PASSWORD defaults	Media Media	M4	H-16
Sin observabilidad distribuida (Micrometer, X-Request-Id)	Media Media	M4	H-17
WS pegajoso + REST libre + Redis como puente	Positivo Positivo	M3, M6	§ 6.2.3

Cuadro 4: Resumen de la Auditoría Manual del Equipo

Conclusiones y Siguientes Pasos

Lo que el sistema hace bien (referencias del equipo, confirmadas por la auditoría cruzada)

Arquitectura por capas limpia y testeable.

Eventos asíncronos para desacoplar push del request.

WebSocket + Redis Pub/Sub bien orquestados.

Web Locks API en el frontend para evitar doble subscribe.

Remount por token en AppLayout y ChatProvider para defensa multi-cuenta.

Seed auto-verificable.

Tests sólidos en chat (concurrencia, expiración, multi-tab, Redis caído).

Lock distribuido en Neo4j para suscripciones push concurrentes.

Nginx con rate limiting diferenciado (auth vs write) y security headers básicos.

Lo que urge corregir antes de cualquier despliegue real

- **Crítica** H-10: Cambiar chmod 644 por chmod 600 en keys-init.
- **Alta** H-11: Forzar corrida de bcrypt en AuthService.login() para evitar user enumeration.
- **Alta** H-12: Agregar Content-Security-Policy estricta en Nginx.
- **Alta** H-28: Implementar PUT /api/auth/password e invalidación de sesiones con tokenVersion en Neo4j.

Lo que urge corregir antes de mostrar a usuarios reales

- **Alta** H-18: Devolver LoginResponse con Profile embebido en register.
- **Alta** H-19: Persistencia de drafts en PostComposer.
- **Alta** H-05: Refactor de PostService (PostCreator + colaboradores).

Lo que se puede hacer en sprints siguientes

- **Media** H-06, H-07, H-08: Limpiar acoplamientos menores.
- **Media** H-14, H-15, H-16, H-17: Endurecimientos de seguridad.
- **Media** H-20, H-21, H-22, H-23, H-24: Fricciones de UX.
- **Media** H-25, H-26, H-27: Tests faltantes.

Documentación recomendada

Una línea en el README: ”Code in English, user strings in Spanish” (H-09).

Procedimiento de rotación de claves JWT/VAPID (H-10).

Política de CSP y qué orígenes push están permitidos (H-12).

Procedimiento de recuperación ante caída de Redis: el usuario debe saber que el chat ”se retrasa” hasta recargar historial (M6 § 6.2.2).

Firmado: Auditoría cruzada sobre la auditoría del equipo Fecha: 2026-11 Alcance: Estático + revisión de código + verificación de cumplimiento del README. No se ejecutaron pruebas de penetración activas (esos quedan para un Blue Team en un entorno autorizado).